MIGA Project

Cold Atoms Bordeaux

MIGA Controller

Operation, Optimization and Analysis Manual

Technical Reference Manual

Complete reference for the `marker-optimization` branch

Prepared by Yiming MENG

Project MIGA / Cold Atoms Bordeaux

Source revision `f60eef0`

Document date September 1, 2026

Laboratory control software for cold-atom acquisition, sequence generation, synchronized measurements, online optimization and reproducible archive analysis.

Document status and scope

This manual describes the behavior implemented in the local `marker-optimization` checkout at revision `f60eef0`. It is written for experimental operators, analysts and maintainers. The graphical user interface, backend routes, data models, numerical routines and tests were reviewed together; therefore a feature is documented only when an executable code path or test supports it. Values shown in screenshots are examples and are not universal experimental calibrations.

Caution. The controller can compile pulse sequences, trigger laboratory hardware and write DDS tables. Confirm beam states, trigger routing, voltage limits and emergency stop procedures before leaving simulation mode. Never treat a successful web request as proof that the optical or electronic apparatus is safe.

How to use this manual

Operators should begin with Chapters 2, 5 and 16. Users creating parameterized sequences should then read Chapters 4 and 7. Optimization users should read Chapters 8 and 9. Analysts should concentrate on Chapters 6, 12 and 13. Maintainers can use the architecture, API and source-map appendices as an implementation index.

Contents

List of Figures

List of Tables

Chapter 1

System overview

1.1 Purpose

MIGA Controller is a browser-based control and data-acquisition application for cold-atom experiments. It combines sequence parameterization, compilation, hardware triggering, dual-channel acquisition, fit-based and integration-based analysis, live visualization, archiving and offline re-analysis. The `marker-optimization` branch adds two complementary closed-loop workflows: deterministic sequential marker optimization and Ax-based Bayesian optimization. It also includes synchronized master/slave scans, scheduled runs, AC Stark and lock-in modes, Bragg pulse generation, Bragg phase-space analysis, archive collections and overlapping Allan deviation.

1.2 Functional map

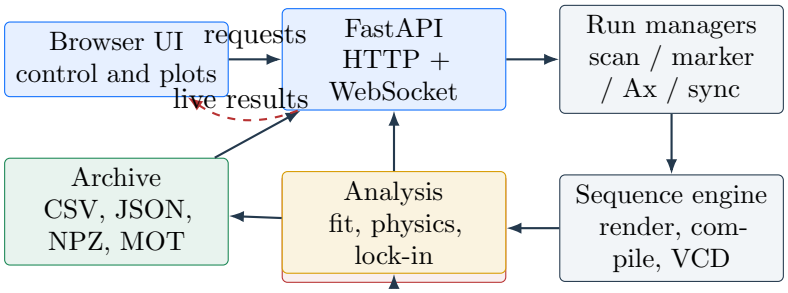

Figure 1.1: End-to-end control and analysis architecture. The WebSocket path returns shot results without blocking the acquisition worker.

1.3 User-facing applications

1.4 Recommended operating sequence

1. Verify paths, hardware platform, network endpoints and analysis constants.
2. Load a sequence and preview the parameterized output.
3. Perform a short simulation scan and inspect both waveform channels.
4. Switch to real hardware only after the trigger and voltage limits are verified.
5. Label the run, execute it, and confirm that its archive contains configuration, results, sequence snapshot and waveform files.
6. Use an optimization workflow only after a bounded manual scan shows that the objective metric is valid over the proposed search interval.

Table 1.1: Primary browser interfaces.

Page	Path	Purpose
Control Console	`/index.html`	Immediate, scheduled and synchronized scans; live fit and waveform plots; sequence upload and scan export.
Marker Workflow	`/marker-optimize.html`	Ordered, fail-fast optimization of named sequence markers.
Optimization	`/optimize.html`	Ax-based online search across discrete `PARAMETER` variables.
Data Archive	`/archive.html`	Run browsing, collections, waveform inspection, recalculation, scan fitting, Allan and Bragg phase analysis.
Settings	`/settings.html`	Hardware, physics, calibration, marker, fit, sync, path and update configuration.

Chapter 2

Installation and startup

2.1 Requirements

The backend requires Python and the packages listed in `requirements.txt`. Real operation additionally requires the local sequence toolchain (normally `tmot4` and `cmot4`), access to the selected acquisition platform, and any site-specific DDS writer used for AC Stark operation. Bayesian optimization requires `ax-platform`. Browser assets are currently loaded from CDNs, so an offline installation must provide cached or locally vendored Vue, Axios, Plotly and Bootstrap assets.

2.2 Launcher

The preferred entry point is:

```
./launch_code
```

The graphical launcher creates `.venv/` when required, checks packages, validates the Ax import path, exposes the `tmot4`/`cmot4` path status, starts or stops the backend and displays the server log. Without a graphical desktop it falls back to the terminal launcher.

Table 2.1: Terminal launcher commands.

Command	Effect
`./start_controller.sh`	Interactive start using the project virtual environment.
`./start_controller.sh --check-only`	Validate the environment without starting.
`./start_controller.sh --status`	Report launcher and server state.
`./start_controller.sh --start-detached`	Start in the background.
`./start_controller.sh --stop`	Request a controlled server stop.

2.3 First-time configuration

Open http://127.0.0.1:8000/settings.html. In **System Paths**, select the `mot4ztex` directory or enter explicit compiler and template paths. In **Network**, choose Red Pitaya or local DAQ and set a timeout longer than the longest expected device response. In **Physics & Gains**, set gains, signal thresholds and calibration constants. Finally, start in simulation mode and run a minimal scan.

Operating procedure: first-start verification

1. Run `./start_controller.sh --check-only` and resolve every missing path.
2. Open the Control Console and confirm the status is **IDLE**.
3. Load the default sequence and check that a preview can be generated.
4. Use a two-point simulation scan and verify that the WebSocket terminal reports both acquisition and analysis completion.
5. Open the archive and confirm that the new run is readable.

Chapter 3

Architecture and execution pipeline

3.1 Application layers

`main.py` creates the FastAPI application, installs CORS, registers the API router, defines `/ws`, and mounts the static frontend last so that it cannot intercept WebSocket requests. The singleton `ExperimentManager` executes blocking hardware work in background threads. A thread-safe bridge submits result broadcasts to the main asynchronous event loop.

Pydantic models in `app/models/schemas.py` validate incoming configurations. Core managers build scan plans, enforce an exclusive run slot, operate hardware and write archives. Analysis modules contain numerical logic without UI state. This separation allows the archive to re-run calculations against stored waveforms.

3.2 Shot lifecycle

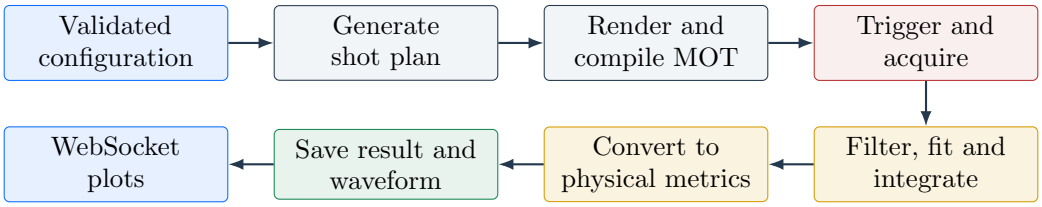

Figure 3.1: Per-shot execution. A failed shot emits an error payload and does not silently become a numeric data point.

3.3 Exclusive run ownership

Standard scans, schedules, marker optimization, Bayesian optimization and synchronized runs share the same experiment hardware. The manager therefore exposes a run slot with an active-mode label. A second workflow is rejected while that slot is owned. Stop requests set workflow-specific flags and then release the slot only after the worker has closed its archive and restored any temporary hardware state.

3.4 Timing from VCD

The sequence compiler produces a VCD representation. `VCDParser` locates the configured launch and trigger channels and determines their physical time separation. This delay enters arrival-time and time-of-flight interpretation. A missing channel is a configuration or sequence error; it should not be replaced with an assumed delay.

Chapter 4

Sequence files and parameterization

4.1 Classic placeholders

Classic mode replaces `PARAMETER0`, `PARAMETER1` and `PARAMETER2` in the active `.mot` template. Each axis can be floating-point or integer. Integer values are rounded at the boundary where the scan plan is built; floating-point values are rounded to six decimal places.

For a step-size axis, the sign of the user step is ignored and the direction follows the relative order of start and stop. The stop point is included when it lies on the grid, within a floating-point tolerance. For an N-point axis, values are linearly spaced:

$$p_i = p_{\min} + \frac{i}{N-1}(p_{\max} - p_{\min}), \qquad i = 0, \ldots, N-1. \tag{4.1}$$

An explicit comma-separated list preserves the specified order after numeric validation.

4.2 Multidimensional scans

Two- and three-dimensional scans use the Cartesian product of the independent axes:

$$\mathcal{P} = \mathcal{P}_0 \times \mathcal{P}_1 \times \mathcal{P}_2. \tag{4.2}$$

Only Standard mode is permitted for multidimensional scans. Each complete grid is repeated according to **Averages**; if **Randomize** is enabled, the expanded shot list is shuffled.

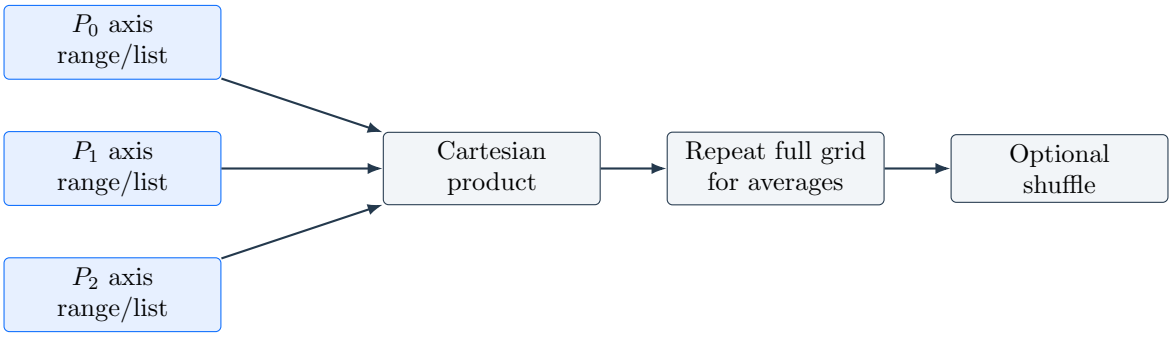

Figure 4.1: Three-dimensional shot-plan construction. Randomization follows averaging expansion.

4.3 Relation modes

For a one-dimensional base value P_0, the controller can derive additional placeholders.

Table 4.1: One-dimensional relation modes.

Mode	Generated parameters	Operational meaning
Standard	$[P_0]$	Independent base scan.
Timing	$[P_0, P_{\text{tot}} - P_0]$	Maintains a fixed total timing; a numeric total is required.
Rabi	$[P_0, P_{\text{tot}} - P_0/2]$	Couples pulse duration to a compensating timing.
Half	$[P_0, P_0/2]$	Supplies the half-value as the second placeholder.
Link	$[P_0, f_1(P_0), f_2(P_0, P_1), \ldots]$	Evaluates ordered formulas with `math`, `numpy`, and preceding parameters.
Bragg Rabi	Derived MOT content	Generates a calibrated Gaussian or Blackman pulse and timing compensation.
Lock-in	ABBA blocks	Repeats two parameter states in the order A–B–B–A.
AC Stark	Interleaved ratio plan	Coordinates MOT placeholders with generated DDS elements.

Caution. Link formulas are evaluated locally with built-ins removed, but they remain configuration code. Review them before use. A formula error aborts plan generation rather than substituting zero.

4.4 Exports

The Control Console can export a single rendered Link or Bragg sequence, or a ZIP of a complete one-dimensional scan. Export paths reuse the same scan-plan logic as execution, so exported values and live-run values remain consistent. A scan ZIP is intentionally bounded to prevent accidental generation of unmanageable archives.

Chapter 5

Main Control Console

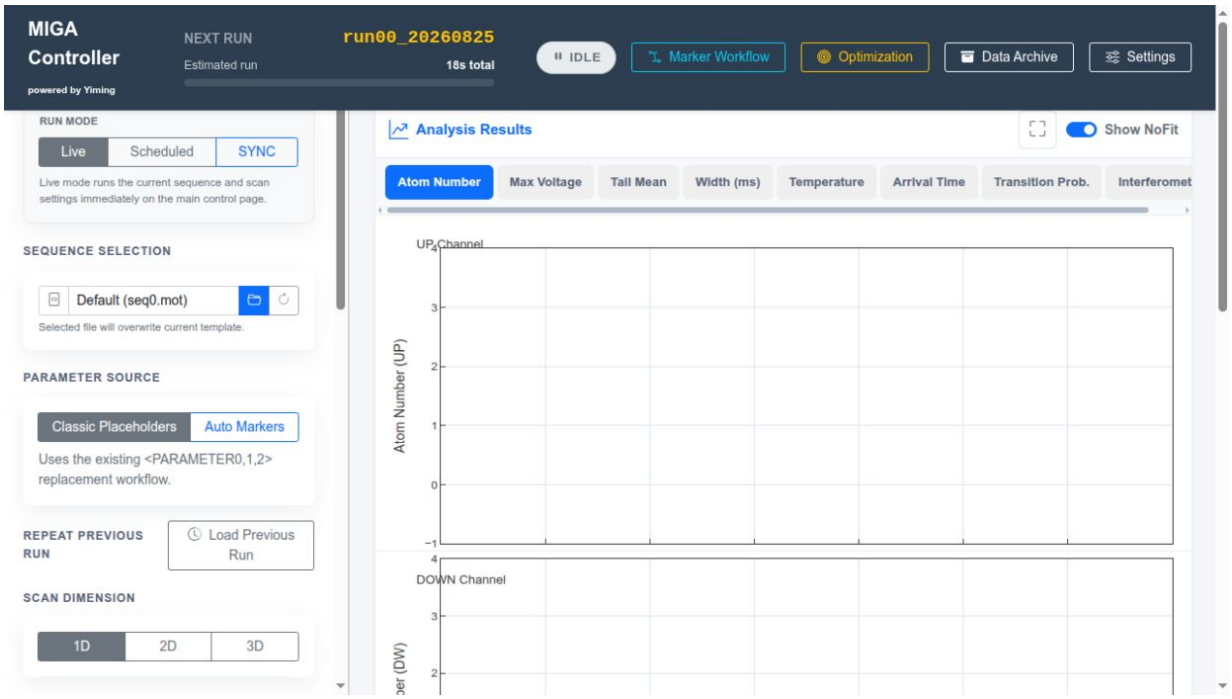

Figure 5.1: Control Console captured from the documented revision. The left pane defines the run; the right pane shows analysis and oscilloscope output.

5.1 Run selection

The header displays the next run ID, estimated duration and current state. **Live** starts the current configuration immediately. **Scheduled** submits one or more saved tasks to the server-side scheduler; after acceptance the browser may close. **SYNC** uses the configured master/slave topology and a shared shot plan.

5.2 Sequence and parameter source

Classic Placeholders uses the rules in Chapter 4. **Auto Markers** replaces selected, validated sequence markers and can preview up to three marker axes. The currently loaded sequence is snapshotted into every new run. **Load Previous Run** restores a saved scan configuration and, for synchronized runs, can restore each node's sequence.

5.3 Live plots

The analysis navigation exposes Atom Number, Max Voltage, Tail Mean, Width, Temperature, Arrival Time, Transition Probability and Interferometer. **Show NoFit** adds integration-based values where available. For multidimensional data the UI supports surface/volume views and axis slices. The oscilloscope panels display raw and fitted UP and DOWN traces; selecting a point associates the aggregate plot with a particular shot.

Operating procedure: run a bounded live scan

1. Confirm **Live**, load the intended sequence and select the parameter source.
2. Set the axis type, range method, start, stop, step/count and numeric type.
3. Choose a relation mode and supply its required parameters.
4. Set both detector fit windows; preview a waveform if the arrival time is uncertain.
5. Set averages, randomization, external trigger and a descriptive run label.
6. Review the estimated shot count and duration, then select **START EXPERIMENT**.
7. Watch the terminal and raw traces. Select **STOP** if saturation, clipping, sequence errors or unexpected hardware states occur.
8. Open the archive after completion and verify the run artifacts.

5.4 Scheduled runs

Tasks can execute sequentially with a configured gap or at scheduled timestamps. The backend persists scheduler state. Before submission, the UI validates each task and estimates its duration. Stopping a schedule requests termination of both the queue and the currently active experiment.

Chapter 6

Signal processing and physical quantities

6.1 Pre-processing and validation

The acquisition driver returns a common time axis and two detector channels. Gain correction is applied before analysis. Values exceeding the configured voltage limit are treated as saturation/error; atom-number conversion requires at least one channel peak to exceed the minimum signal threshold. A fit window selects the time interval used by the nonlinear model. The raw/NoFit path uses the same stored waveform with its configured integration rule.

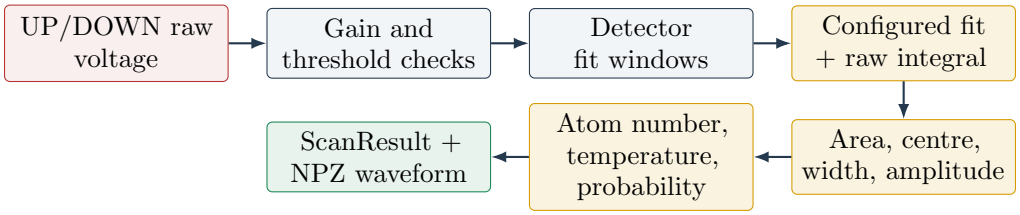

Figure 6.1: Signal-to-physics pipeline. Fit and NoFit calculations share the calibrated input but use different area estimators.

6.2 Waveform models

The built-in time-of-flight models include Gaussian, two skewed/modified Gaussians, generalized Lorentzian and sinc-squared profiles. Custom models are represented by a formula, named parameters, guess expressions, fixed/free flags and semantic roles (amplitude, width, centre and offset). Formula syntax is parsed through a restricted AST before numerical evaluation.

For the Gaussian model,

$$V(t) = V_0 + A \exp\left[-\frac{(t-t_0)^2}{2\sigma^2}\right], \qquad \mathcal{A}_{\mathrm{G}} = |A\sigma|\sqrt{2\pi}. \tag{6.1}$$

Models without the Gaussian-area mode use the baseline-removed numerical integral

$$\mathcal{A}_{\mathrm{fit}} = \left| \int_{t_a}^{t_b} \left[V_{\mathrm{fit}}(t) - V_0 \right], \mathrm{d}t \right|. \tag{6.2}$$

The edge-line NoFit method estimates a linear baseline from the first and last n points and integrates the residual. The legacy method preserves the original controller behavior.

6.3 Atom-number calibration

The conversion factor K is derived in SI units from detection velocity v_d, wavelength λ, light-sheet height h_s, transimpedance G_T, collection efficiency η, photodiode responsivity ρ, saturation ratio s, detuning Δ and natural linewidth Γ:

$$K = \frac{v_d \lambda}{h_s G_T \eta h c \rho} \frac{2\left[1 + s + (2\Delta/\Gamma)^2\right]}{\Gamma s}. \tag{6.3}$$

All positive calibration inputs are checked for finiteness and positivity.

With crosstalk coefficients α and β, DOWN/UP sensitivity ratio R and integrated areas $\mathcal{A}_{U,D}$,

$$N_{F=2} = (\mathcal{A}_U - \alpha \mathcal{A}_D) K, \tag{6.4}$$

$$N_{F=1} = (\mathcal{A}_D - \beta \mathcal{A}_U) R K, \tag{6.5}$$

$$P_{F=i} = 100 \frac{N_{F=i}}{N_{F=1} + N_{F=2}}. \tag{6.6}$$

6.4 Interferometer correction

The interferometer correction uses independent coefficients α_I, β_I and γ_I. Let $n_1 = N_{F=1}$ and $n_2 = N_{F=2}$. Then

$$D = (\beta_I - \alpha_I)(1 + \gamma_I), \tag{6.7}$$

$$N_1 = \frac{\beta_I n_1 - (1 - \beta_I + \gamma_I) n_2}{D}, \tag{6.8}$$

$$N_2 = \frac{(1 - \alpha_I + \gamma_I) n_2 - \alpha_I n_1}{D}, \tag{6.9}$$

$$P_i = 100 \frac{N_i}{N_1 + N_2}. \tag{6.10}$$

If D or the corrected total is numerically zero, the software returns unavailable or zero probabilities rather than dividing by a small number.

6.5 Arrival time and temperature

For launch velocity v_0, target height z and gravitational acceleration g,

$$t_{\mathrm{arr}} = \frac{v_0 \pm \sqrt{v_0^2 - 2 g z}}{g}. \tag{6.11}$$

A negative discriminant means the trajectory does not reach the target. The time-of-flight temperature inferred from temporal width is

$$T_{\mu\mathrm{K}} = 10^6 \frac{m_{\mathrm{Rb87}}}{k_B} \left(\frac{v_0}{t_{\mathrm{flight}}} - g \right)^2 \sigma_t^2. \tag{6.12}$$

The caller must supply σ_t in the declared units; the implementation converts milliseconds when requested.

Chapter 7

Sequence markers and visual editing

7.1 Marker model

A marker replaces a semantic quantity rather than a positional placeholder. Each definition contains a unique ID, display name, type, decimal precision, hard minimum and maximum, default scan start/stop/step, axis method, expected command/channel and an optional compensation flag. Supported types are `dds_element`, `dac_value`, `duration` and `digital_state`.

Marker metadata is embedded as comments in the `.mot` document and is also stored in sequence-specific profiles. The profile key prevents definitions from leaking across unrelated sequences. The editor preserves the source encoding and line-ending convention.

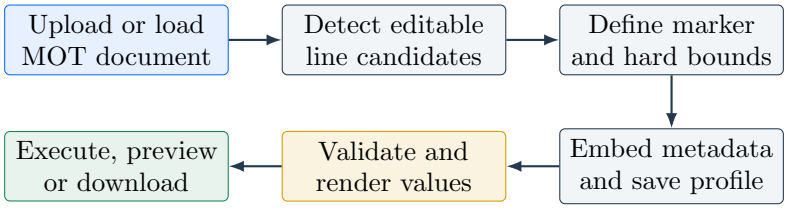

Figure 7.1: Visual marker authoring and use. Marker validation is repeated at render time.

7.2 Candidate and compensation logic

The inspector detects editable DDS element numbers, DAC values, durations and digital states from executable lines. Expected command and channel fields provide structural validation. A compensation marker can update a second line when the scanned value must preserve a total duration. Digital-state markers do not form numeric axes; each workflow step selects Keep current, ON or OFF.

Operating procedure: create a marker profile

1. Open **Settings** > **Sequence Markers** and load the intended `.mot` file.
2. Inspect candidate lines and select the exact quantity to control.
3. Enter a descriptive name and stable uppercase ID.
4. Choose the marker type, precision, hard bounds and safe default scan.
5. Set expected command/channel values and a compensation line if required.
6. Add the marker, inspect the highlighted source, and save the document profile.
7. Download the marked file and keep it with the experimental sequence repository.
8. Preview values at both hard limits before permitting hardware execution.

Caution. Renaming or moving an executable line can invalidate its embedded marker signature. Reinspect the document after any manual sequence edit. Never widen hard limits only to make a workflow pass validation.

Chapter 8

Sequential marker optimization

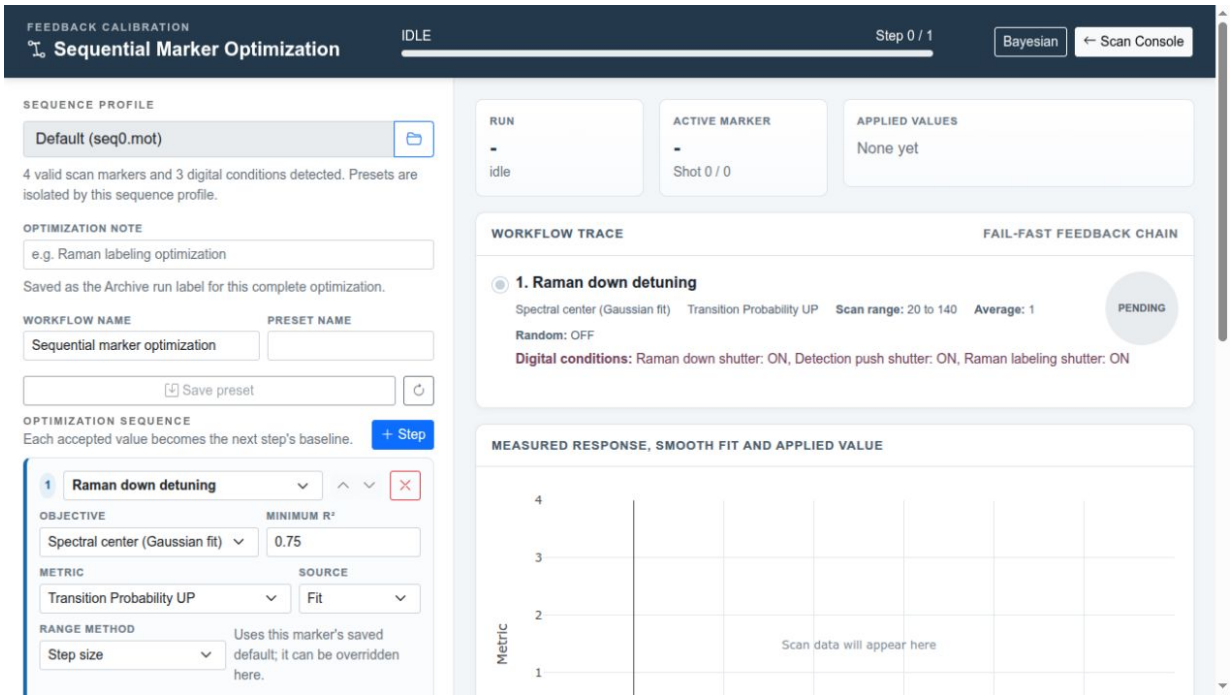

Figure 8.1: Sequential Marker Optimization. Each accepted value is applied to the baseline used by all subsequent steps.

8.1 Feedback-chain design

Marker optimization is deterministic and ordered. A workflow step scans one numeric marker while applying specified digital states. Repeated measurements are aggregated, the chosen objective is evaluated, and the accepted value is written into the in-memory baseline. If fitting or safety validation fails, execution stops before the next step.

8.2 Objectives

Measured maximum and minimum choose an actually sampled point. Ties select the smaller axis value. A selected endpoint is rejected because it indicates that the optimum may lie outside the scan.

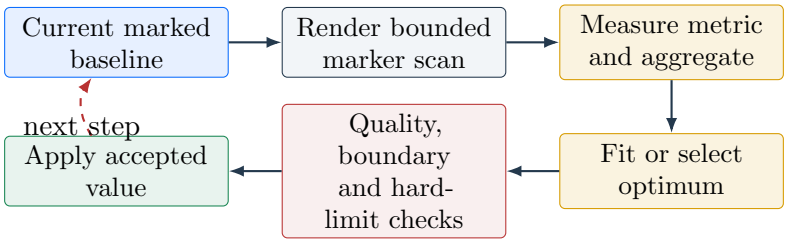

Figure 8.2: Fail-fast sequential marker optimization.

Spectral-centre fitting requires at least five distinct points and uses

$$y(x) = c + A \exp\left[-\frac{(x - x_0)^2}{2\sigma^2}\right]. \tag{8.1}$$

The accepted value is the fitted centre x_0.

First-π Rabi fitting requires at least six positive duration values and uses

$$y(x) = c + Ae^{-x/\tau} \sin^2\left(\frac{\pi x}{2t_\pi}\right). \tag{8.2}$$

The accepted value is t_π. Both fitted objectives require

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \geq R^2_{\min}. \tag{8.3}$$

The continuous optimum must remain inside the scan. DDS elements and durations are rounded to integers; DAC values are rounded to the marker precision. This representable value is validated again against the scan and hard limits.

8.3 Repeated measurements

For m repeats, the workflow stores every shot and reports

$$\bar{y} = \frac{1}{m} \sum_{j=1}^{m} y_j, \tag{8.4}$$

$$s = \sqrt{\frac{1}{m} \sum_{j=1}^{m} (y_j - \bar{y})^2}, \tag{8.5}$$

$$\mathrm{SEM} = \frac{s}{\sqrt{m}}. \tag{8.6}$$

The implementation uses population standard deviation for repeat aggregation. The chosen metric can come from the fitted or NoFit result.

Operating procedure: run a sequential marker workflow

1. Load a saved marker sequence profile and confirm the detected marker count.
2. Enter a workflow name and run label; optionally load or save a preset.
3. Add steps in physical dependency order.
4. For each step, choose marker, objective, metric, source, scan range, average count, randomization and minimum R^2.
5. Set every digital condition explicitly when the safe state is known.
6. Review the complete workflow trace and confirm that every range stays within the stored marker hard limits.
7. Start the workflow and inspect the measured curve, residuals and applied value.
8. On completion, download the optimized sequence, workflow CSV and JSON report; also verify them in the archive.

8.4 Failure behavior

A missing metric, invalid marker, insufficient points, failed shot, poor R^2, out-of-range fit, endpoint optimum or user stop prevents the current value from becoming a downstream baseline. The archive retains completed steps and the error context for diagnosis.

Chapter 9

Bayesian optimization

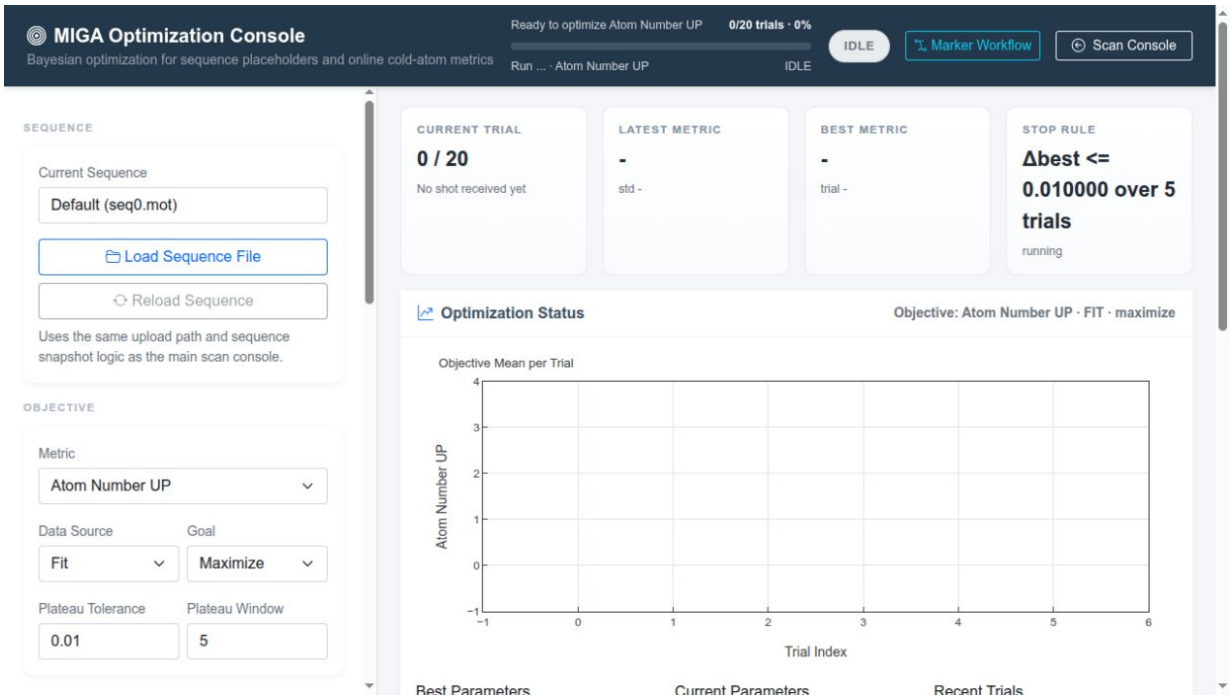

Figure 9.1: Bayesian Optimization Console. It shows trial progress, the objective history, the current/best parameter sets and the active stop rule.

9.1 Search space and generation strategy

The Bayesian workflow uses Ax. Each configured `PARAMETER`n is an ordered discrete choice list generated from lower bound, upper bound and step. Integer variables are rounded; floating variables are rounded to six decimals. A single variable is limited to 5000 grid values. The optional initial guess is snapped to the closest legal grid point.

The Ax client configures an experiment, an initialization budget and the scalar objective `optimization_score`. If no initial guess exists and the requested random budget is zero, at least one initialization trial is retained. Measured value is also reported as a tracking metric where the installed Ax API supports it.

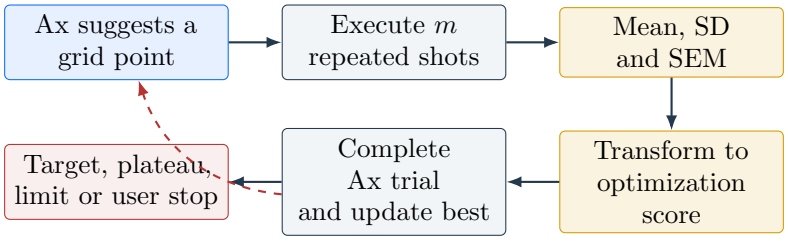

Figure 9.2: Bayesian optimization loop. Hardware evaluation remains the dominant operation; Ax only selects the next legal parameter tuple.

9.2 Objective scoring

Let $\bar{y}$ be the repeat mean. The internal score maximized by Ax is

$$q(\bar{y}) = \begin{cases} \bar{y}, & \text{maximize}, \\ -\bar{y}, & \text{minimize}, \\ -|\bar{y} - y_\star|, & \text{target}. \end{cases} \tag{9.1}$$

Supported metrics include UP/DOWN atom number, amplitude, temperature, arrival time, transition probability and interferometer populations/probabilities. The source can be Fit or NoFit.

Target mode stops when $|\bar{y} - y_\star|$ is no greater than the target tolerance. Maximize and minimize modes can stop on a plateau. If q_k^{best} is the best score after trial k, a window w stops when

$$q_k^{\text{best}} - q_{k-w}^{\text{best}} \leq \epsilon_{\text{plateau}}. \tag{9.2}$$

Operating procedure: run Bayesian optimization

1. Perform a manual bounded scan for each candidate variable.
2. Load the same sequence into the Optimization Console.
3. Select metric, Fit/NoFit source and maximize/minimize/target goal.
4. Add unique `PARAMETER` indices with legal bounds, grid step and type.
5. Choose repeat count, maximum trials and initial random trials.
6. Configure either target tolerance or plateau tolerance/window.
7. Start the run and monitor objective dispersion as well as the mean.
8. Export `optimization_history.csv`, `optimization_report.json` and the best rendered sequence.

Note. A low mean uncertainty does not prove that the global optimum was found. Search quality depends on the bounded grid, metric stability, initialization and trial budget.

Chapter 10

Bragg, AC Stark and lock-in workflows

10.1 Calibrated Bragg pulse generation

The Bragg generator writes channel 32 with a sampled Gaussian or Blackman envelope. For a Gaussian FWHM w,

$$\sigma = \frac{w}{2\sqrt{2\ln 2}}, \qquad t \in [0, 8\sigma]. \tag{10.1}$$

For a Blackman envelope, total duration is $w/0.405$. Samples use the configured clock resolution (default 0.2 µs). Two off samples are included, so the timing compensation is

$$P_1 = t_{\text{base}} - (N_{\text{sample}} + 2)\Delta t. \tag{10.2}$$

Negative compensation and more than two million samples are rejected.

Normalized optical power is converted to voltage by inverting a generalized logistic calibration. For voltage v,

$$P(v) = L + (U - L)\left[1 + e^{-k(v - v_0)}\right]^{-\nu}. \tag{10.3}$$

The selected linear voltage interval is renormalized to $[0, 1]$. Requests below the off threshold map to −3 V; operating voltages must remain within −3 V to 3 V.

10.2 AC Stark centering

The AC Stark workflow validates an uploaded `ad9958` XML table and appends one DDS element per requested Raman power ratio $r = P_{R1}/P_{R2}$. At constant total power,

$$P_{R1} = P_{\text{tot}}\frac{r}{1+r}, \qquad P_{R2} = P_{\text{tot}}\frac{1}{1+r}. \tag{10.4}$$

Each power is inverted through its own generalized-logistic amplitude calibration and rounded to an integer in $[0, 1023]$. The actual powers and ratio are recalculated from the rounded amplitudes and archived. XML element numbers and counts are limited to 500.

The workflow interleaves left and right sequence points, writes and verifies the generated DDS table, performs the measurements, then attempts to restore the original table. Both the original and generated XML plus the ratio plan are stored with the run.

Caution. If DDS restoration reports an error, stop dependent experiments and verify the physical DDS state with the site procedure before running another sequence.

10.3 ABBA digital lock-in

Lock-in mode creates complete four-shot blocks with states A–B–B–A. The reference vector is

$$\mathbf{r} = (+1, -1, -1, +1). \tag{10.5}$$

For signals $S_1, \ldots, S_4$,

$$S_A = \frac{S_1 + S_4}{2}, \qquad S_B = \frac{S_2 + S_3}{2}, \tag{10.6}$$

$$X = \frac{S_1 - S_2 - S_3 + S_4}{4}, \tag{10.7}$$

$$R = \frac{S_A - S_B}{S_A + S_B}, \tag{10.8}$$

$$E = S_A - 2S_B, \qquad E_N = \frac{S_A - 2S_B}{S_A + 2S_B}. \tag{10.9}$$

Only complete blocks with the expected states and references contribute. Each supported Fit and NoFit metric receives block values plus mean, sample standard deviation and SEM.

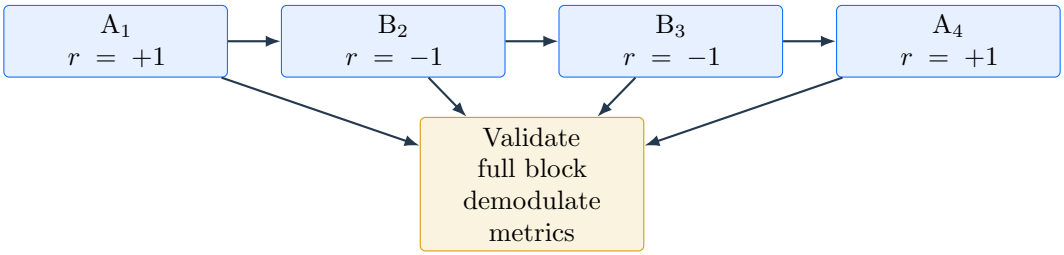

Figure 10.1: ABBA lock-in block and reference sequence.

Chapter 11

Synchronized multi-controller operation

11.1 Roles and authorization

A controller is configured as Standalone, Master or Slave. The master stores a list of enabled slaves with node IDs, names, base URLs and per-node sequences. Requests can carry a shared token; a slave may also restrict the allowed master IP. The **Test** action checks reachability before a run.

11.2 Prepare, start and pair

The master generates the complete scan once and serializes it as a shot plan. Each slave receives the run ID, master/slave IDs, scan configuration, shot plan and sequence content (text or base64 for binary-safe transfer). Every slave must report ready before any start command is sent. Slaves start first; the master begins after the configured delay.

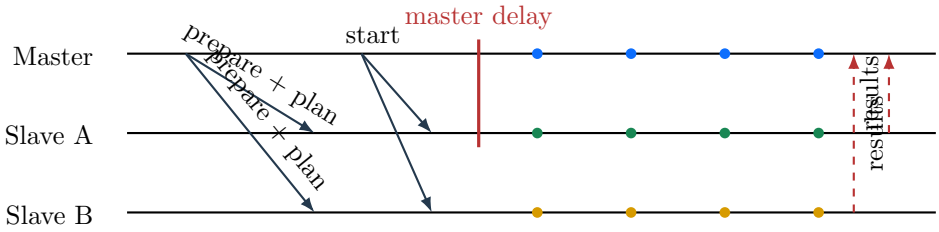

Figure 11.1: Master/slave synchronization. Pairing uses logical shot index, not network arrival order.

Master and slave results are keyed by `sync_shot_index`. A pair is emitted only when both sides exist and neither contains an error. The UI can display per-node curves or differential paired results and replay completed points.

11.3 Archive replication

Each node first writes a normal local archive. A ZIP bundle excludes temporary files and nested sync copies. SHA-256 checksums protect transfer integrity. Extraction rejects path traversal, stages files in a temporary directory, requires both `results.csv` and `config.json`, and atomically replaces the previous node copy. The final master archive contains `sync_nodes/<node-id>/` and a sync manifest. Replication is bidirectional; failed transfers can be retried from the archive interface.

Operating procedure: run a synchronized scan

1. Configure one Master and one or more Slaves with the same shared token.
2. Test every enabled slave from **Settings** > **SYNC**.
3. In the Control Console select **SYNC**, load the master sequence and each slave sequence, then configure a bounded scan.
4. Set master delay according to the physical trigger arrangement.
5. Start and confirm that all nodes enter ready/running states.
6. Monitor paired count, per-node progress and differential plots.
7. After completion, open the master archive, select every node and verify archive replication status. Retry incomplete transfers.

Chapter 12

Data archive and re-analysis

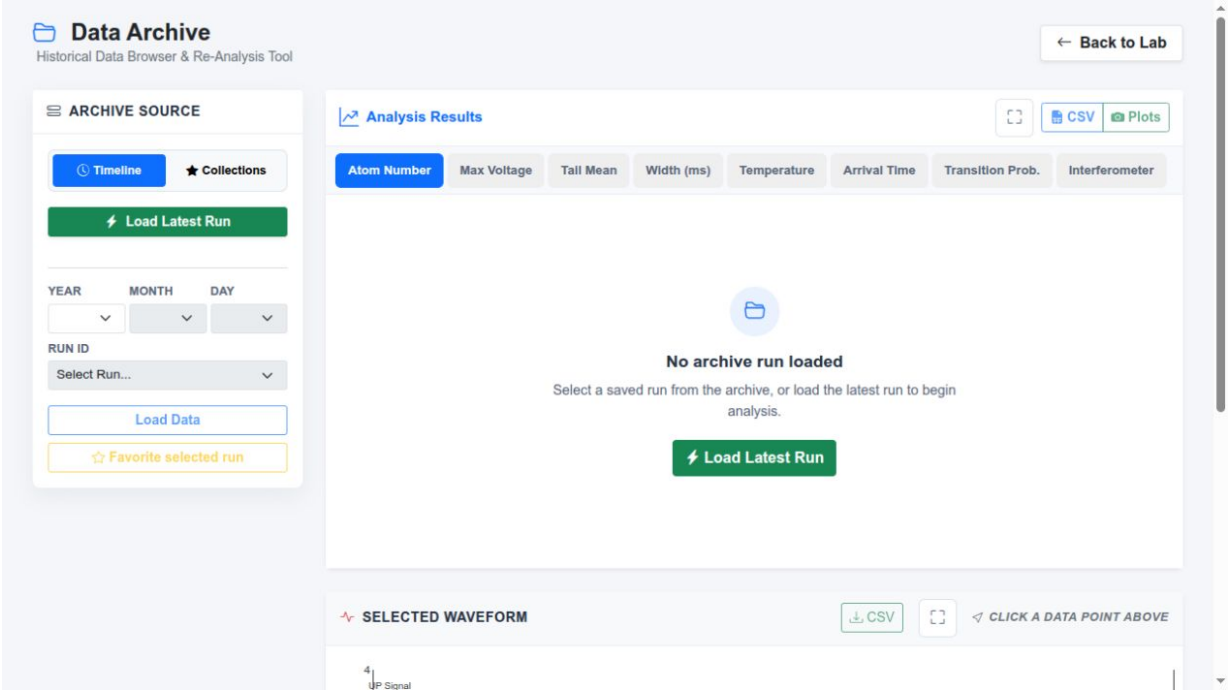

Figure 12.1: Data Archive before a run is loaded. Timeline and Collections provide two paths to the same immutable run data.

12.1 Archive structure

Runs are organized by year, month and day. A typical directory is

```
Data_log/2026/08/25/run00_20260825/
  config.json
  results.csv
  sequence.mot
  waveforms/step_0001.npz
  sync_manifest.json              # synchronized runs only
  sync_nodes/<node-id>/...         # replicated node archives
```

The configuration is a reproducibility snapshot. `results.csv` provides fast aggregate loading. Each NPZ stores time, raw UP/DOWN, fitted UP/DOWN and fit-window arrays.

12.2 Timeline and collections

Timeline browsing selects year, month, day and run. Collections are SQLite-backed folders containing favourites. A favourite can have an alias, note, preview metric and preview step. Batch operations support moving or removing selected favourites without changing the underlying experiment directory.

12.3 Re-analysis

Recalculation loads the raw waveform, normalizes new analysis settings, reapplies gain and fit windows, runs the selected fit model and recomputes physics metrics. The display can switch between saved and recalculated results. **Overwrite** updates calculated archive products only when the user explicitly requests it; raw NPZ waveforms remain the primary evidence.

12.4 Scan fitting

Aggregate curves can be fitted over a selected x range. Built-in scan models are Gaussian, Lorentzian, Rabi oscillation, atom-interferometer fringes, Bragg fringes and Custom. The model editor defines formula, parameter guesses, fixed flags and semantic roles. Fit curves are evaluated on 32–4000 points for display. A custom model should first be tested on a copy of a run.

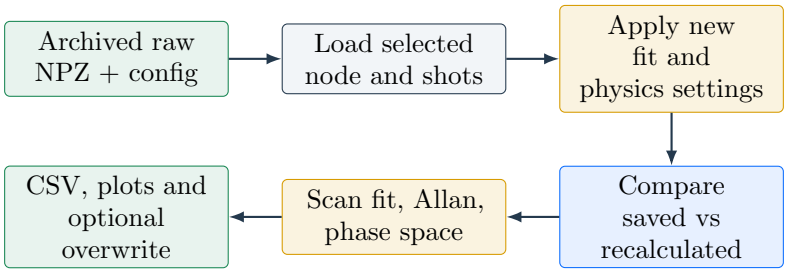

Figure 12.2: Archive re-analysis preserves raw evidence while allowing new derived products.

Chapter 13

Bragg fringe, phase space and stability

13.1 Bragg fringe acceleration fit

With $x = T^2$ expressed in μs^2, the model is

$$y(x) = C + A\cos(\omega x + \phi_0), \qquad k_{\text{eff}} = \frac{4\pi n}{\lambda}. \tag{13.1}$$

For a fixed candidate ω, the offset and cosine/sine coefficients form a linear least-squares problem. The implementation scans a bounded frequency grid, refines the best candidates with scalar minimization, and rejects frequencies above the sampling limit or the acceleration projection of standard gravity. It then reports

$$a = \frac{\omega\, 10^{12}}{k_{\text{eff}}}, \qquad \theta = 10^3 \arcsin\left(\frac{a}{g_0}\right) \text{ mrad}. \tag{13.2}$$

Mid-fringe locations satisfy

$$x_j = \frac{\pi/2 + j\pi - \phi_0}{\omega}, \qquad \Delta x_{\text{mid}} = \frac{\pi}{\omega}. \tag{13.3}$$

Valid Bragg fit results can be saved as named reusable phase calibrations.

13.2 Phase-space conversion

For a measured value y_i, calibration amplitude A and corrected offset C, the normalized signal is clipped for inversion:

$$z_i = \text{clip}\left(\frac{y_i - C}{A}, -1, 1\right). \tag{13.4}$$

Because $\arccos z$ is multi-valued, the software enumerates both signs and neighboring 2π cycles, then chooses the phase nearest the prediction $\phi_i^{\text{pred}} = \omega P_{0,i} + \phi_0$ or a fixed mid-fringe reference. The wrapped deviation is $\delta\phi_i = \text{wrap}(\phi_i - \phi_i^{\text{pred}})$.

Sensitivity is $|\sin\phi_i^{\text{pred}}|$. A point is high quality only when it is inside the user-defined mid-fringe band and its normalized signal was not clipped. Optional reference-first correction shifts C by the mean residual of the first N points.

13.3 Archive interferometer phase and SYNC A/C optimization

13.3.1 Monotonic half-fringe phase conversion

The Archive interferometer-phase view uses each node's own effective Bragg calibration. For a SYNC archive this is normally the calibration snapshot saved by that node; an archive phase-reference override, when present, becomes the effective calibration. The selected signal field (for example `intf_p1` or `intf_p1_nofit`), slope, ω, ϕ_0 and reference T^2 therefore remain independent for Master and Slave.

For measured signal $S_{i,k}$ from node i and shot k, define

$$u_{i,k} = \frac{S_{i,k} - C_i}{A_i}. \tag{13.5}$$

Unlike the general phase-space display above, Archive interferometer phase does not clip an out-of-envelope value: $|u_{i,k}| > 1$ makes the shot invalid. A saved monotonic-slope choice selects one π-wide half-fringe. With

$$s_i = \begin{cases} +1, & \text{negative-slope half-fringe,} \\ -1, & \text{positive-slope half-fringe,} \end{cases} \tag{13.6}$$

the reported phase relative to the calibration reference is

$$\Delta\phi_{i,k} = s_i \arccos(u_{i,k}) - s_i \arccos\left[\cos\left(\omega_i T_{i,\mathrm{ref}}^2 + \phi_{0,i}\right)\right]. \tag{13.7}$$

SYNC rows are paired by logical shot index and matching $P_0 = T^2$. For selected Reference and Target nodes the differential sequence is

$$d_k = \Delta\phi_{\mathrm{Target},k} - \Delta\phi_{\mathrm{Reference},k}. \tag{13.8}$$

The ordinary series view wraps the difference to $(-\pi, \pi]$. Optimization unwraps the chronological sequence because the expected physical differential phase is an unknown constant and a wrap boundary must not create artificial noise.

13.3.2 Why optimize A and C

Equation (13.7) shows that changing only ω or ϕ_0 changes a constant reference term and therefore cannot change standard deviation or first-order Allan deviation. Amplitude and offset change the per-shot nonlinear inversion. Their local sensitivities are

$$\frac{\partial\Delta\phi}{\partial A} = s\frac{u}{A\sqrt{1-u^2}}, \qquad \frac{\partial\Delta\phi}{\partial C} = s\frac{1}{A\sqrt{1-u^2}}. \tag{13.9}$$

At mid-fringe $u \simeq 0$, C is directly constrained while the first-order sensitivity to A is weak. The saved fringe curve and parameter prior are consequently essential when estimating A from a mid-fringe-only SYNC sequence.

The optimizer varies four independent parameters,

$$\mathbf{p} = (A_R, C_R, A_T, C_T), \tag{13.10}$$

starting from the Reference and Target nodes' respective effective calibrations. It keeps ω, ϕ_0, reference T^2, monotonic slope and signal-source selection fixed. Reversing the displayed difference swaps Reference and Target roles but never causes the nodes to share calibration parameters. The Archive page submits the selected nodes and controls to `/archive/sync-phase-calibration-optimize`; the server reads the paired raw signal fields and both effective calibrations from the selected archive.

13.3.3 Objective and constraints

The selectable SYNC metric $m(d)$ is

$$m(d) = \begin{cases} \sigma_{\mathrm{A},1}(d), & \text{Allan n=1,} \\ \mathrm{std}(d), & \text{Standard deviation,} \\ \sqrt{\lambda\sigma_{\mathrm{A},1}^2(d) + (1-\lambda)\,\mathrm{std}^2(d)}, & \text{Combined,} \end{cases} \tag{13.11}$$

where λ is **Allan Weight**. The first-order value is

$$\sigma_{\mathrm{A},1}(d) = \sqrt{\frac{1}{2(N-1)}\sum_{k=1}^{N-1}(d_{k+1}-d_k)^2}. \tag{13.12}$$

The normalized optimization loss is

$$L = \left(\frac{m(d)}{m(d_0)}\right)^2 + w_f L_{\mathrm{fringe}} + w_p L_{\mathrm{prior}} + L_{\mathrm{invalid}}, \tag{13.13}$$

where d_0 is the sequence produced by the original calibrations. For node i, candidate A_i, C_i are compared with that node's own saved fitted curve $(x_j, y_j^{\mathrm{fit}})$:

$$\widehat{y}_{i,j} = C_i + A_i\cos(\omega_i x_j + \phi_{0,i}), \tag{13.14}$$

$$L_{\mathrm{fringe}} = \left(\frac{\mathrm{RMSE}_R}{A_{R,0}}\right)^2 + \left(\frac{\mathrm{RMSE}_T}{A_{T,0}}\right)^2. \tag{13.15}$$

Thus **Fringe Weight** w_f controls how strongly the optimized curves must remain close to the two saved fitting curves. The current constraint uses saved `fit_x/ fit_y`; it does not refit the original raw fringe shots.

For an **A/C Bounds** fraction b, the hard parameter intervals are

$$A_i \in [A_{i,0}(1-b), A_{i,0}(1+b)], \tag{13.16}$$

$$C_i \in [C_{i,0} - bA_{i,0}, C_{i,0} + bA_{i,0}]. \tag{13.17}$$

The soft prior is the mean squared displacement normalized by these allowed scales,

$$L_{\mathrm{prior}} = \frac{1}{4}\sum_q \left(\frac{p_q - p_{q,0}}{bA_{i(q),0}}\right)^2. \tag{13.18}$$

Prior Weight w_p therefore controls direct preference for the original A, C values, whereas **Fringe Weight** controls agreement with the original curves. Candidate parameters that make a normalized signal leave $[-1, 1]$ receive an overflow and invalid- shot penalty; invalid shots are not silently removed to improve the objective.

Recommended initial controls are $b = 10\%$, $w_f = 1$ and $w_p = 0.05$. Reducing the weights allows more aggressive SYNC-noise reduction; increasing them makes the result more conservative. Do not set both weights to zero for mid-fringe data. A parameter reaching its hard boundary is reported and normally indicates that the bound, data model or calibration quality needs review rather than automatic acceptance.

Operating procedure: preview a joint SYNC A/C optimization

1. Load a replicated SYNC run in the Data Archive and select **Interferometer Phase**.
2. Select exactly two hosts. The first is Reference and the second Target; use **Reverse Difference** when required.
3. Set the inclusive P_0 range to the chronological shots that should constrain the constant differential phase.

4. In **Joint A/C calibration optimization**, choose **Allan n=1**, **Standard deviation** or **Combined**; then set bounds and constraint weights.
5. Click **Optimize A/C**. Inspect parameter changes, valid-pair count, boundary warnings, saved-curve RMSE and the mean-removed before/after differential plot.
6. The optimized per-node phases immediately feed the existing Phase Series, Allan plot and Sequence Statistics. Changing Allan order or P_0 filtering recalculates those views from the preview values.
7. Click **Reset preview** to restore the archived phases.

Caution. Optimization results are preview-only. They do not overwrite an archived run and do not save or activate a Settings calibration. A small Allan or standard deviation alone is insufficient evidence of a valid calibration: also inspect curve agreement, parameter movement, valid-shot count and boundary warnings.

13.4 Overlapping Allan deviation

Allan analysis is available only for non-randomized one-dimensional scans. After optional inclusive P_0 filtering, for averaging order n define adjacent block means

$$\bar{y}_i^{(n)} = \frac{1}{n} \sum_{j=0}^{n-1} y_{i+j}. \tag{13.19}$$

The overlapping Allan deviation is

$$\sigma_y(n) = \sqrt{\frac{1}{2M} \sum_{i=1}^{M} \left(\bar{y}_{i+n}^{(n)} - \bar{y}_i^{(n)} \right)^2}, \tag{13.20}$$

where only windows whose $2n$ samples are finite contribute. The maximum order is half the filtered sequence length. The UI supports absolute or mean-normalized values, linear or logarithmic order axes, and displays valid-window count, mean, RMS and sample standard deviation for Fit and raw channels.

Chapter 14

Settings, calibration and maintenance

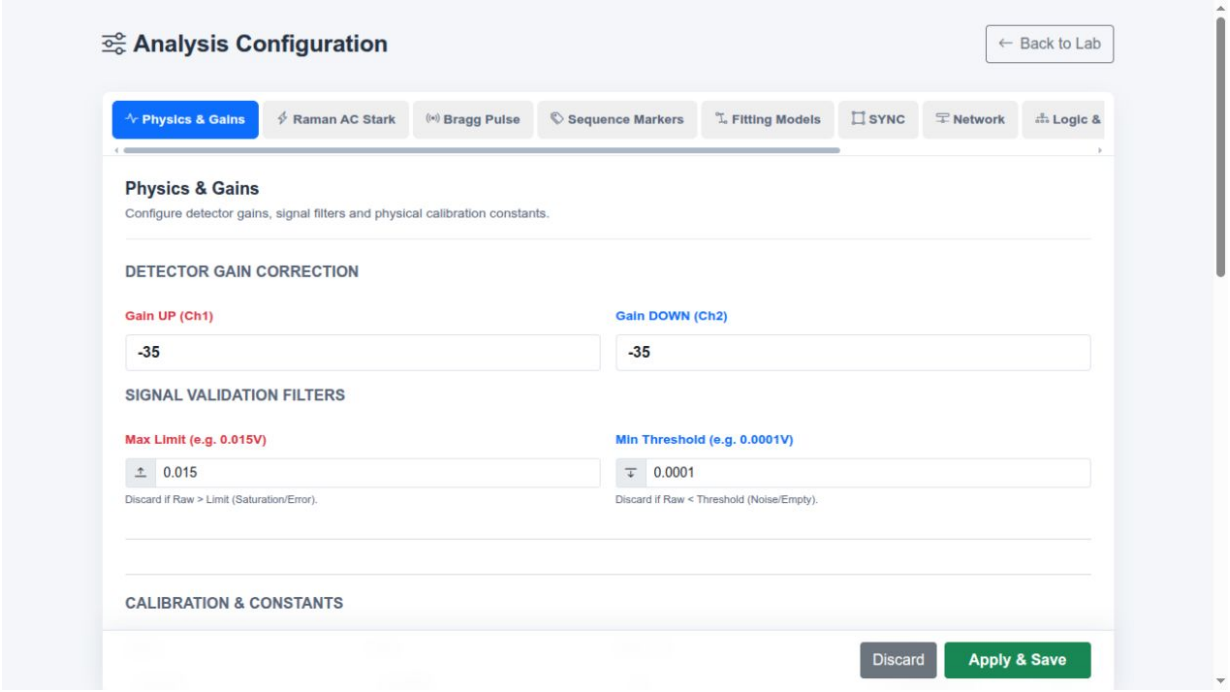

Figure 14.1: Settings interface. Tabs separate physical calibration from execution, network and maintenance concerns.

Table 14.1: Settings tabs and responsibilities.

Tab	Responsibility
Physics & Gains	Detector gains, validation limits, K inputs, crosstalk, geometry, launch velocity, decimation and interferometer coefficients.
Raman AC Stark	DDS writer path and four independent R1/R2 amplitude-to-power calibrations for UP and DOWN groups.
Bragg Pulse	Channel-32 voltage-to-power logistic calibration, linear interval and off threshold, with raw and normalized preview plots.
Sequence Markers	Visual marker editor, saved documents, definitions, hard bounds, compensation and embedded metadata.
Fitting Models	Atom-area method, baseline points, waveform model formulas, guesses, roles and fixed/free parameters.

Tab	Responsibility
SYNC	Standalone/Master/Slave role, node name, token, allowed IP and slave endpoints.
Network	Acquisition platform, Red Pitaya addresses, DAQ rate and timeout.
Logic & Channels	Launch/trigger channel IDs and linked-mode timing constant.
System Paths	`tmot4`, arguments, `cmot4`, default template and DDS writer.
System Update	Repository URL, branch, fetch status and controlled update action.

Settings are saved to `user_settings.json`. The manager refreshes runtime settings from disk before starting optimization. Keep a versioned laboratory copy of calibration values, but do not commit host-specific credentials or tokens.

Operating procedure: calibration change control

1. Export or copy the existing settings and record the current run reference.
2. Change one calibration family at a time.
3. Use preview plots and a known reference sequence to validate the direction and range of the response.
4. Record the calibration source, date, apparatus state and valid interval.
5. Run a small validation scan and retain its archive before using the calibration in optimization or scheduled operation.

Chapter 15

Data products and exports

Table 15.1: Principal reproducibility artifacts.

Artifact	Contents
`config.json`	Complete run configuration and analysis/settings snapshot.
`results.csv`	Per-shot parameters, Fit/NoFit metrics, mode metadata and errors.
`sequence.mot`	Exact sequence snapshot used by the run.
`waveforms/step_NNNN.npz`	Time axis, raw and fitted channels, fit windows.
Marker workflow CSV/JSON	Step definitions, repeated values, fits, selected/applied values and error state.
Optimized marker sequence	Final baseline after all accepted marker steps.
`optimization_history.csv`	Trial parameters, mean, SD, SEM, score and best-so-far.
`optimization_report.json`	Optimization configuration and final status.
Best sequence	Classic placeholder template rendered with best Bayesian parameters.
AC Stark XML and plan	Original/generated DDS tables and requested/actual power ratios.
`sync_manifest.json`	Node identities, progress, paired data and replication state.

To reproduce a numerical result, retain the raw waveform, configuration snapshot, exact sequence, software revision and any external hardware calibration or firmware version that is not stored by the controller. A plot image or CSV alone is insufficient.

Chapter 16

Troubleshooting

Table 16.1: Symptom-oriented troubleshooting guide.

Symptom	Likely cause / check	Corrective action
Controller does not start	Wrong interpreter, missing package, stale launcher state or occupied port.	Run `--check-only` and `--status`; inspect `.launcher_state/server.log`; stop the known prior process before changing ports.
Ax import failure	Backend started outside `.venv` or `ax-platform` is missing.	Start through `launch_code` or `start_controller.sh`; repair the virtual environment rather than installing Ax only for root/system Python.
Compiler path error	`tmot4`, `cmot4` or template path is missing/not executable.	Use **System Paths** auto-fill, then verify each resolved file and execute permission.
Compilation fails	Invalid MOT syntax, unresolved placeholder, or unsupported marker rendering.	Download/preview the rendered sequence, run the compiler manually, and inspect the first reported source line.
VCD timing unavailable	Launch/trigger channel IDs do not exist in compiled VCD.	Match **Logic & Channels** to the actual sequence channel numbers and recompile.
DAQ timeout	Wrong IP/platform, network route, too-short timeout or device busy.	Test the device independently, confirm the selected platform, increase timeout within laboratory policy and retry a single simulation shot first.
Waveform rejected	Peak exceeds voltage limit or both channels remain below minimum.	Inspect raw traces; correct optical/electronic gain and thresholds. Do not simply widen limits around a saturated detector.
Fit fails	Empty/incorrect fit window, too few samples, poor initial guess or unsuitable model.	Display raw data, move the window, select a physically suitable model and test with fixed/simple parameters.
NoFit disagrees strongly	Baseline drift or inappropriate area method.	Compare legacy and edge-line integration, increase baseline points cautiously, and inspect the waveform edges.
Link formula error	Invalid expression/name or dependency order.	Use only P_0, earlier P_i, `math` and `np`; evaluate formulas one at a time.
Marker not detected	Target line is not executable, signature changed or profile differs.	Reopen the original MOT in the visual editor, inspect candidates and save a new profile.
Embedded metadata invalid	Manual comment corruption or conflicting definitions.	Keep a copy, remove/recreate the affected marker through the editor, then download the repaired document.

Symptom	Likely cause / check	Corrective action
Compensation rejected	Second line missing, wrong type/channel or computed value violates its representation.	Inspect both marked lines and preview both ends of the scan.
Marker optimum on boundary	Scan does not bracket a measured extremum.	Expand or shift the range within hard limits and repeat; do not accept the endpoint automatically.
Marker fit below R^2	Low SNR, wrong model, insufficient points or multimodal response.	Inspect residuals, increase repeats/points, improve the scan range or choose measured max/min when physically justified.
Bayesian objective unavailable	Selected metric is `None`/nonfinite for the chosen Fit/NoFit source.	Verify a manual shot and select a metric produced by that analysis path.
Bayesian run stops early	Target reached, plateau rule triggered, user stop or evaluation error.	Inspect `optimization_report.json` `stop_reason` and trial history.
DDS XML invalid	Wrong root, duplicate/out-of-range elements or more than 500 entries.	Repair the source table; never bypass XML validation.
AC Stark power invalid	Requested power cannot be inverted or rounded amplitude leaves calibrated range.	Reduce total/range or replace the calibration with measured valid data.
DDS write/verify fails	Writer path, permissions, hardware access or verification command.	Stop the workflow, inspect captured stdout/stderr and verify hardware state before retry.
Bragg pulse rejected	Nonpositive FWHM, invalid calibration, overlong pulse or too many samples.	Check shape/FWHM, base timing, clock resolution and linear voltage interval.
Slave not ready	URL unreachable, role mismatch, token/IP rejection or busy run slot.	Use **Test**; inspect slave status/log; make roles and credentials consistent.
Missing sync pairs	Node shot error, index mismatch or incomplete result polling.	Compare per-node progress and errors using the shared `sync_shot_index`.
Archive checksum mismatch	Incomplete/corrupt transfer.	Keep local archives, retry sync replication, and do not manually install the unverified ZIP.
Allan mode unavailable	Run is randomized or not one-dimensional.	Use the original shot sequence only for a non-random 1D run; do not reorder data to force availability.
Phase points clipped	$\lvert(y - C)/A\rvert > 1$ because calibration or measurement is inconsistent.	Refit the fringe, inspect offset/amplitude drift and exclude clipped points from precision statistics.
Archived waveform missing	NPZ was not written, moved or belongs to another node.	Select the correct sync node, inspect the run directory and use aggregate data only with the missing-evidence limitation recorded.
WebSocket disconnected	Browser/network interruption or backend restart.	Refresh after the backend is stable; query experiment status before starting another run.

16.1 Diagnostic evidence to collect

Record the run ID and label, UTC/local time, active mode, sequence snapshot, configuration, server log excerpt, full error message, relevant raw waveform and hardware state. Do not include shared tokens or unrelated credentials in a support bundle.

Appendix A

Quick-start checklists

A.1 Routine live scan

☐ Correct sequence and parameter source loaded.
☐ Scan range and relation mode previewed.
☐ Fit windows cover both expected detector arrivals.
☐ Gains, threshold and voltage limit match current apparatus state.
☐ Hardware/simulation mode and trigger selection verified.
☐ Run label describes the experimental purpose.
☐ Archive checked after completion.

A.2 Optimization readiness

☐ Manual scan brackets the response.
☐ Objective metric is finite for every representative point.
☐ Hard limits reflect hardware safety limits.
☐ Repeat count resolves expected noise.
☐ Stop rules and maximum trial budget reviewed.
☐ Safe digital conditions explicitly selected.
☐ Generated sequence and report retained.

A.3 Synchronized run readiness

☐ Exactly one master; all other controllers configured as slaves.
☐ Tokens, allowed IPs and base URLs verified.
☐ All slave connection tests pass.
☐ Node sequences correspond to the shared shot plan.
☐ Master delay matches physical triggering.
☐ All node archives and checksums verified after completion.

Appendix B

Formula and unit reference

Table B.1: Core symbols.

Symbol	Meaning	Unit / convention
$\mathcal{A}_{U,D}$	Detector signal area	V s
K	Area-to-atom conversion	atoms per V s
α, β, R	Detection crosstalk/sensitivity factors	dimensionless
σ_t	Temporal fit width	seconds internally
v_0	Launch velocity	$\mathrm{m\,s^{-1}}$
z	Detection height	m
A, C, ϕ_0	Fringe amplitude, offset and phase	metric units, metric units, rad
ω	Bragg phase rate versus T^2	rad per $\mu\mathrm{s}^2$
k_{eff}	Effective Bragg wave vector	$\mathrm{rad\ m^{-1}}$
R^2	Marker fit coefficient of determination	dimensionless
s, SEM	Repeat dispersion and standard error	objective units
n	Allan averaging order	shots

Appendix C

API endpoint reference

Table C.1: API groups in `app/api/routes.py`.

Group	Principal endpoints and purpose
Experiment	`/experiment/start`, `/experiment/stop`, `/experiment/status`, sequence upload, template snapshot, previous-run preset.
Sequence markers	Inspect upload/text, annotate, update, remove, preview, download and saved marker document/profile operations.
Exports	Single/scan Bragg MOT and Link MOT/ZIP generation; DDS table upload/status.
Schedule	Start, stop and status for persisted task queues.
Bayesian optimization	Objective catalog, start, stop, status and artifact download.
Marker optimization	Catalog, workflow start/stop/status, presets and artifact download.
Sync	Configuration, connectivity test, master start/stop/status, slave health/prepare/ start/stop/status and archive transfer.
Settings/system	Simulation mode, all/analysis settings, folder browser and controlled repository update status/fetch/apply.
Fitting	Default waveform models, default scan models and saved custom scan model.
Archive	Tree, load, sequence/waveform, recalculate, overwrite, Allan, scan fit, Bragg phase space, reusable calibrations, SYNC A/C phase-calibration preview and sync retry.
Collections	Folder and favourite create/update/delete plus batch operations.
WebSocket	`/ws` broadcasts shot, trial, sync pair, completion and error payloads.

Appendix D

Source-code map

Table D.1: Implementation map for maintainers.

Path	Responsibility
`main.py`	FastAPI, WebSocket bridge and static mounting.
`app/api/routes.py`	HTTP endpoints, response packaging and archive/export calls.
`app/models/schemas.py`	Validated request and settings models.
`app/core/experiment_manager.py`	Run slot, scan plan, sequence execution, acquisition and result processing.
`app/core/sequence_markers.py`	Marker inspection, embedded definitions, validation and rendering.
`app/core/marker_optimization_manager.py`	Sequential workflows, fitting, baseline application and artifacts.
`app/core/optimization_manager.py`	Ax client, online trials, stop rules and reports.
`app/core/sync_manager.py`	Master/slave protocol, pairing and archive replication.
`app/core/data_manager.py`	Run creation, CSV/JSON/NPZ persistence.
`app/core/data_loader.py`	Archive loading, recalculation, Allan and sync nodes.
`app/core/pulse_generator.py`	Calibrated Gaussian/Blackman Bragg pulses.
`app/drivers/dds_table.py`	DDS XML, Raman calibration inversion and write/verify.
`app/analysis/fitting.py`	Waveform/scan models, safe formulas and Bragg fringe.
`app/analysis/physics.py`	K, atom numbers, probabilities, arrival and temperature.
`app/analysis/lock_in.py`	ABBA validation and demodulation statistics.
`app/analysis/phase_space.py`	Bragg phase inversion and quality statistics.
`app/analysis/interferometer_phase.py`	Monotonic half-fringe Archive phase conversion and reference subtraction.
`app/analysis/phase_calibration_optimization.py`	Joint SYNC Reference/Target A, C optimization, constraints, preview phases and diagnostics.
`static/*.html`	Vue interfaces and Plotly visualization.

Path	Responsibility
`tests/`	Regression evidence for markers, optimization, sync, archive, fitting, calibration, exports and settings.

Appendix E

Safety and validation limits

- Scan axes must contain numeric values; point counts and steps must be positive where required. Marker scans require at least two distinct points.
- Marker values must remain inside stored hard limits. Fitted marker optima must also remain inside the scanned interval after representation rounding.
- DDS amplitudes are restricted to $0 \ldots 1023$; element numbers/counts to $0 \ldots 500$ and at most 500 elements.
- Bragg voltage calibration operates within $-3\,\mathrm{V}$ to $3\,\mathrm{V}$; off is $-3\,\mathrm{V}$; normalized power cannot exceed one.
- Bayesian variables have unique parameter indices and at most 5000 discrete values each. At least one variable and one trial are required.
- Allan deviation is defined here only for the original order of non-randomized 1D scans. Invalid windows are omitted and their count is reported.
- SYNC A/C optimization must retain a fringe constraint or parameter prior, especially for mid-fringe data where A is weakly identifiable. Preview results are not persisted automatically.
- Sync archives are accepted only after checksum and safe-extraction validation.
- Software limits supplement rather than replace independent hardware interlocks.

Revision note

This edition was generated from the local `marker-optimization` branch at commit `f60eef0` and rendered on September 1, 2026. Screenshots were captured from the same checkout using its simulation configuration. When software behavior changes, update the revision identifier, regenerate screenshots, rerun the regression suite and visually inspect the complete PDF before issue.